

Prototypespesifikasjon: JDD kunnskapsdatabase

2026-09-22 · @Someone

Instruks til Claude

Bygg en kjørbart prototype som starter med `docker compose up` og viser hvordan kundeservice-e-poster blir til anonymiserte, godkjente artikler i en kunnskapsdatabase.

- Les hele dokumentet før du skriver kode, og følg spesifikasjonen. Er noe uklart, spør i stedet for å gjette.
- Prototypen er en demo for et studentprosjekt (IS-219, UiA) for bedriften JDD Utstyr. Prioriter at dataflyten er synlig og lett å forklare, fremfor ytelse og skalering.
- All data er oppdiktet. Ingen ekte kundedata skal inn i systemet.
- Prototypen skal fungere uten API-nøkkel (regelbasert modus). Med `ANTHROPIC_API_KEY` satt brukes KI til uttrekk og ekstra anonymisering.
- Brukergrensesnittet skal være på norsk (bokmål).
- Hold koden enkel og kommentert, slik at studentene kan forklare den ved presentasjon.

Kontekst: JDD Utstyr selger bilbelysning, lysbjelker, bilpleie og tilbehør til bil, tilhenger, traktor og landbruksmaskiner. Løsninger på kunde problemer blir i dag liggende i e-poster og kommer ikke inn i kunnskapsdatabasen. En ansatt i kundesenteret skal alltid godkjenne innhold før det lagres.

Arkitektur

Fem Docker-tjenester: en test-e-postserver, en database, et API med innhentingsjobb, et webgrensesnitt og en databaseviser.

```
flowchart LR
    S[seed-skript] -->|SMTP| M[mailpit<br/>test-innboks]
    M -->|REST-polling| W[api: ingest-worker]
    W --> DB[(postgres)]
    API[api: FastAPI] --> DB
    UI[web: frontend] --> API
    AD[adminer] --> DB
```

Seed-skriptet sender oppdiktete e-poster til Mailpit, som etterligner kundesenterets innboks. Worker henter dem og kjører pipelinen.

Tjeneste	Image / teknologi	Port	Oppgave
mailpit	axllent/mailpit	8025 (web), 1025 (SMTP)	Test-innboks som simulerer Microsoft-e-posten
db	postgres:16 med pg_trgm	5432	Kunnskapsdatabase og hendelseslogg
api	Python 3.12, FastAPI, SQLAlchemy	8000	REST-API og bakgrunnsjobb (ingest-worker)
web	Enkel HTML/JS servert av nginx	8080	Grensesnitt for ansatte og chatbot-demo
adminer	adminer	8081	Vise tabellene direkte under presentasjon

Miljøvariabler i `.env` (med `.env.example`): `DATABASE_URL`, `MAILPIT_URL`, `ANTHROPIC_API_KEY` (valgfri), `LLM_MODEL` (standard `claude-sonnet-5`), `POLL_INTERVAL_SECONDS` (standard 10), `DUPLICATE_THRESHOLD` (standard 0.4).

Datamodell

Fem tabeller. Rå e-post lagres aldri; bare anonymisert tekst går videre.

```
CREATE EXTENSION IF NOT EXISTS pg_trgm;

CREATE TABLE products (
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  sku         VARCHAR(50) UNIQUE NOT NULL,
  name        VARCHAR(200) NOT NULL,
  category    VARCHAR(100) NOT NULL -- f.eks. Lysbjelke, Arbeidslys,
  Bilpleie, Tilhenger
);

-- Anonymisert kopi av hver e-posttråd som er behandlet
CREATE TABLE email_items (
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  source_msg_id VARCHAR(200) UNIQUE NOT NULL, -- Mailpit-ID, hindrer
  dobbel behandling
  anonymized_text TEXT NOT NULL,
  status        VARCHAR(20) NOT NULL,          -- received, anonymized,
  extracted, proposed, failed
  created_at    TIMESTAMP NOT NULL DEFAULT now()
);
```

```

CREATE TABLE knowledge_articles (
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id  INT REFERENCES products(id),
  title       VARCHAR(200) NOT NULL,
  problem     TEXT NOT NULL,
  solution    TEXT NOT NULL,
  tags        VARCHAR(300),
  version     INT NOT NULL DEFAULT 1,
  approved_by VARCHAR(100) NOT NULL,
  approved_at TIMESTAMP NOT NULL DEFAULT now(),
  updated_at  TIMESTAMP NOT NULL DEFAULT now()
);

-- KI-forslag som venter på godkjenning
CREATE TABLE proposals (
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email_item_id  INT REFERENCES email_items(id),
  product_id    INT REFERENCES products(id),
  title         VARCHAR(200) NOT NULL,
  problem       TEXT NOT NULL,
  solution      TEXT NOT NULL,
  tags          VARCHAR(300),
  proposal_type VARCHAR(20) NOT NULL, -- new, update
  matched_article_id INT REFERENCES knowledge_articles(id),
  similarity     NUMERIC(4,3),
  status         VARCHAR(20) NOT NULL DEFAULT 'pending', -- pending,
  approved, rejected
  reviewed_by    VARCHAR(100),
  reviewed_at    TIMESTAMP,
  created_at     TIMESTAMP NOT NULL DEFAULT now()
);

-- Hendelseslogg som viser dataflyten steg for steg
CREATE TABLE pipeline_events (
  id          INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email_item_id INT REFERENCES email_items(id),
  step        VARCHAR(30) NOT NULL, -- received, anonymized, extracted,
  duplicate_check, proposed, approved, rejected
  detail      TEXT,
  created_at  TIMESTAMP NOT NULL DEFAULT now()
);

CREATE INDEX idx_articles_problem_trgm ON knowledge_articles USING gin
(problem gin_trgm_ops);

```

Skjemaet opprettes av `db/init.sql` ved første oppstart.

Pipeline

Workeren kjører hvert `POLL_INTERVAL_SECONDS` og behandler nye meldinger i fem steg. Hvert steg skriver en rad i `pipeline_events`.

```
sequenceDiagram
    participant M as Mailpit
    participant W as Worker
    participant K as KI / regler
    participant DB as Postgres
    W->>M: Hent nye meldinger
    W->>W: Anonymiser tekst
    W->>DB: Lagre email_item
    W->>K: Trekk ut produkt, problem, løsning
    W->>DB: Duplikatsjekk mot artikler
    W->>DB: Lagre proposal (pending)
```

- 1. Innhenting.** Hent meldinger fra Mailpit REST-API. Hopp over meldinger der `source_msg_id` allerede finnes. Hver e-post inneholder kundens henvendelse og kundeservicens svar i samme tråd.
- 2. Anonymisering (før noe lagres).** Regex erstatter e-postadresser med `[E-POST]`, norske telefonnumre (8 siffer, ev. med +47) med `[TELEFON]`, ordrenumre (mønster `ORD-` eller `Ordre nr`) med `[ORDRE]`, og navn i hilsen/signatur (`Hei <Navn>`, `Mvh <Navn>`) med `[NAVN]`. Postnummer og adresselinjer fjernes. I KI-modus sendes den regelvaskede teksten i tillegg til modellen med instruks om å fjerne gjenværende personopplysninger.
- 3. Uttrekk.** KI-modus: kall Anthropic Messages API og be om kun JSON med feltene `sku`, `title`, `problem`, `solution`, `tags`. Produktet skal være et av SKU-ene i `products`; ellers `null`. Regelmodus: finn SKU eller produktnavn i teksten, bruk kundens første avsnitt som `problem` og kundeservicens svar som `solution`. Formuler problem og løsning generelt, uten referanse til kunden.
- 4. Duplikatsjekk.** Beregn `similarity(problem, $1)` med `pg_trgm` mot artikler for samme produkt. Er høyeste verdi $\geq$ `DUPLICATE_THRESHOLD`, blir forslaget `update` med `matched_article_id`; ellers `new`.
- 5. Forslag.** Lagre i `proposals` med status `pending`. Ingenting skrives til `knowledge_articles` før en ansatt godkjenner.

Ved feil i et steg: sett `email_items.status = 'failed'`, logg feilen i `pipeline_events`, og fortsett med neste melding.

API-endepunkter

FastAPI med automatisk dokumentasjon på `/docs` . Ingen innlogging; ansattnavn sendes med i forespørselen.

Metode	Sti	Beskrivelse
GET	<code>/api/proposals?status=pending</code>	Liste over forslag som venter, med matchet artikkel hvis <code>update</code>
GET	<code>/api/proposals/{id}</code>	Ett forslag med anonymisert e-posttekst og hendelseslogg
POST	<code>/api/proposals/{id}/approve</code>	Body: <code>reviewer</code> , ev. redigerte <code>title</code> , <code>problem</code> , <code>solution</code> , <code>tags</code> . Oppretter ny artikkel, eller oppdaterer matchet artikkel og øker <code>version</code>
POST	<code>/api/proposals/{id}/reject</code>	Body: <code>reviewer</code> , <code>reason</code>
GET	<code>/api/articles?q=&product=</code>	Søk i godkjente artikler (pg_trgm + ILIKE på tittel, problem, tags)
GET	<code>/api/articles/{id}</code>	Én artikkel
GET	<code>/api/products</code>	Produktliste
POST	<code>/api/chat</code>	Body: <code>question</code> . Finner de 3 mest relevante artiklene og svarer. KI-modus: modellen svarer kun ut fra artiklene og sier fra hvis svaret ikke finnes. Regelmodus: returnerer beste artikkel. Svaret viser alltid hvilke artikler det bygger på
GET	<code>/api/events?limit=50</code>	Siste hendelser for dataflyt-visningen
GET	<code>/api/stats</code>	Antall e-poster, forslag per status, artikler
POST	<code>/api/demo/reset</code>	Tømmer alt unntatt produkter og seed-artikler, for ny demo

Brukergrensesnitt

En enkel webside med fire faner. Ren HTML, CSS og JavaScript uten byggesteg, så nginx kan servere filene direkte.

1. **Dataflyt** (startside). Viser tellere fra `/api/stats` og en live-oppdatert liste fra `/api/events` (poll hvert 3. sekund). Hvert steg har egen farge, så man kan følge en e-post fra mottatt til godkjent under demo.
2. **Godkjenning** (kundesenter). Liste over ventende forslag. Klikk åpner en visning med anonymisert e-post til venstre og redigerbart forslag til høyre. Ved `update` vises eksisterende artikkel ved siden av, med likhetsprosent. Knapper: Godkjenn, Avvis (med begrunnelse). Felt for ansattnavn huskes i nettleseren.
3. **Kunnskapsbase** (kundesenter). Søkefelt og produktfilter, resultatliste med tittel, produkt, versjon og godkjent av.
4. **Chatbot** (kunde). Enkelt chatvindu som kaller `/api/chat` og viser kildeartiklene under svaret.

Krav: norsk tekst, lesbar på mobil, tydelig merking av at all data er oppdiktet.

Testdata

Alle produkter, kunder og e-poster er oppdiktet. Byttes ut med anonymiserte eksempler fra JDD når de kommer.

- **db/seed.sql** : 8–10 fiktive produkter i kategoriene Lysbjelke, Arbeidslys, Kabelsett, Bilpleie og Tilhenger (SKU-format `DEM0-001`), og 3 godkjente artikler slik at duplikatsjekk kan vises.
- **seed/emails/*.json** : 10 e-posttråder med feltene `from`, `subject`, `customer_message`, `support_reply`. Hver tråd har oppdiktet navn, e-post, telefon og ordrenummer, slik at anonymiseringen synes.
- Minst 3 e-poster skal likne en eksisterende artikkel (gir `update`), og minst 1 skal mangle gjenkjennelig produkt (gir `product_id = null`).
- Eksempler på problemtyper: lysbjelke som flimrer ved motorstart, feil kabelsett til en lysbjelke, arbeidslys som dugger innvendig, tilhengerlys som ikke virker etter montering, bilpleieprodukt som etterlater striper.

seed/send_emails.py : sender e-postene til Mailpit via SMTP, én og én med 2 sekunders pause, slik at dataflyten kan følges live. Kjøres med `docker compose run --rm seed`.

Filstruktur og oppstart

```
jdd-kunnskapsdatabase/
  docker-compose.yml
  .env.example
  README.md           # oppstart og demo-manus på norsk
  db/
```

```

init.sql
seed.sql
api/
  Dockerfile
  requirements.txt
app/
  main.py          # FastAPI-app og ruter
  db.py
  models.py
  worker.py        # polling av Mailpit og pipeline
  anonymize.py
  extract.py       # KI-modus og regelmodus
  duplicates.py
  chat.py
tests/
  test_anonymize.py
  test_duplicates.py
web/
  Dockerfile       # nginx
  index.html
  app.js
  style.css
seed/
  Dockerfile
  send_emails.py
emails/

```

Oppstart:

1. `cp .env.example .env`
2. `docker compose up --build`
3. `docker compose run --rm seed`
4. Åpne `http://localhost:8080` (app), `http://localhost:8025` (innboks) og `http://localhost:8081` (database).

Akseptansekriterier

- ☐ `docker compose up --build` starter alle tjenester uten feil, med og uten `ANTHROPIC_API_KEY`.
- ☐ Etter seed vises hver e-post som hendelser i Dataflyt-fanen innen 15 sekunder.
- ☐ Ingen navn, e-postadresser, telefonnumre eller ordrenumre fra testdata finnes i databasen (sjekkes med test).
- ☐ Minst én e-post gir forslag av typen `update` med synlig likhetsprosent.

- ☐ Godkjenning oppretter eller oppdaterer artikkel; avvisning lagrer begrunnelse. Ingenting lagres i `knowledge_articles` uten godkjenning.
- ☐ Godkjent artikkel kan finnes i søk og brukes i chatbot-svar med kildehenvisning.
- ☐ `/api/demo/reset` gjør det mulig å kjøre demoen på nytt.
- ☐ README forklarer oppstart og et demo-manus på 5 minutter.